

JavaScript Key Learnings — Question 3

OJT Assessment Prep — ES6 Classes, this Keyword, Context Loss & .bind()

1. ES6 Classes

A **class** is a blueprint for creating objects with shared properties and methods.

```
class Car {
  constructor(brand) {    // runs when you do: new Car(...)
    this.brand = brand;   // stores data on the object
  }

  honk() {                // a method – function inside the class
    console.log(this.brand + " beep!");
  }
}

const myCar = new Car("Toyota");
myCar.honk(); // "Toyota beep!"
```

■ **constructor()** runs automatically when you call **new ClassName()**.

2. The this Keyword

this refers to whoever is calling the function at that moment. Inside a class, **this** = the object instance.

```
class Person {
  constructor(name) {
    this.name = name;    // this = the new Person object
  }
  greet() {
    console.log(this.name); // this = Person instance ■
  }
}
```

3. Context Loss — The Core Problem

When you pass a class method as an event listener, **this stops pointing to your class**. This is called **context loss**.

■ **The broken version:**

```
class LikeComponent {
  constructor() {
    this.likes = 0;
    btn.addEventListener("click", this.registerClick);
    //                                ↑ passing the function, not calling it
  }

  registerClick() {
```

```
    this.likes++; // ■ BREAKS! this = the DOM button, not the class!
  }
}
```

Where	What this refers to
Inside class method (normal call)	■ The class instance
Inside event listener callback	■ The DOM element

4. .bind() — Fixing Context Loss

.bind(thisValue) creates a new version of a function where **this is permanently locked** to whatever you pass in.

```
function greet() {
  console.log(this.name);
}
const person = { name: "John" };
const locked = greet.bind(person);
locked(); // "John" — this is always person, no matter what
```

Applied to fix context loss in a class:

■ **Without bind — context lost:**

```
btn.addEventListener("click", this.registerClick);
```

■ **With bind — context locked:**

```
btn.addEventListener("click", this.registerClick.bind(this));
```

5. Scope — let vs this inside Constructor

This is the most common bug when writing classes. Where you declare a variable decides where it can be used.

■ **Wrong — local variable dies after constructor:**

```
constructor(id) {
  let btn = document.getElementById(id); // local — gone after constructor
}
registerClick() {
  btn.innerText = "..."; // ■ btn is undefined here!
}
```

■ **Correct — store on this so it lives everywhere:**

```
constructor(id) {
  this.btn = document.getElementById(id); // lives on the object
}
registerClick() {
  this.btn.innerText = "..."; // ■ accessible anywhere in the class
}
```

6. id String vs DOM Element

A common confusion — an **id string** and a **DOM element** are not the same thing!

```
"likeBtn" // just a string – no methods
document.getElementById("likeBtn") // the actual DOM element – has methods
```

Always convert the id string to an element using **getElementById** before calling methods on it.

7. Script Tag Position — Why Your Button May Not Work

If JS runs before the HTML is loaded, **getElementById** returns **null** and nothing works.

■ **Wrong — script in <head>, button not loaded yet:**

```
<head>
  <script src="script.js"></script>  <!-- runs before button exists! -->
</head>
<body>
  <button id="likeBtn">Like (0)</button>
</body>
```

■ **Correct — script at bottom of <body>:**

```
<body>
  <button id="likeBtn">Like (0)</button>
  <script src="script.js"></script>  <!-- button exists before JS runs -->
</body>
```

■■ Final Correct Code

```
class LikeComponent {
  constructor(id) {
    this.btn = document.getElementById(id); // string → element, stored on this
    this.likes = 0; // counter initialized
    this.btn.addEventListener("click",
      this.registerClick.bind(this) // this locked to class instance
    );
  }

  registerClick() {
    this.likes++; // increments the class counter
    this.btn.innerText = `Like(${this.likes})`; // updates button text
  }
}

let a = new LikeComponent("likeBtn"); // instantiate the class
```

■ Golden Rules to Remember

Rule	Remember
Context Loss	Always use .bind(this) when passing class methods as callbacks

Scope	Use this.x to share data across methods; let x is local only
id vs element	Always call getElementById() to convert string to element
Script position	Always place script tag at the bottom of body